perplexity

Сделай сохранение результатов в CSV файл

Добавим сохранение результатов в CSV через `pandas.DataFrame.to_csv`. Это самый удобный способ: сохраняет и время, и все контуры, с заголовками и без индекса.[1] [2]

Что добавить

В код нужно:

1. Импортировать `pandas`.
2. Создать `DataFrame` из результатов.
3. Сохранить в `output/model_results.csv`.

Обновлённый фрагмент

Добавьте в начало:

```
import pandas as pd
```

И добавьте функцию:

```
def save_results_to_csv(sol, filename='output/model_results.csv'):
    df = pd.DataFrame({
        'time': sol.t,
        'S': sol.y[^0],
        'O': sol.y[^1],
        'M': sol.y[^2],
        'P': sol.y[^3],
        'A': sol.y[^4],
        'D': sol.y[^5],
    })
    df.to_csv(filename, index=False)
    print(f"Результаты сохранены в {filename}")
```

Как встроить в `main()`

Обновите `main()` так:

```
def main():
    os.makedirs('output', exist_ok=True)
    sol = run_simulation()
```

```
    # Сохранение в CSV
    save_results_to_csv(sol, 'output/model_results.csv')

    plot_2d(sol)
    plot_3d(sol)
    animate(sol)
    print_key_findings(sol)
```

Полный обновлённый код

```
import os
import sys
import subprocess
import importlib
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib import animation
from scipy.integrate import solve_ivp
from dataclasses import dataclass, field
from typing import List, Tuple

# ------------------------------
# Проверка и установка библиотек
# ------------------------------

REQUIRED = ['numpy', 'scipy', 'matplotlib', 'pandas']

def ensure_dependencies():
    for pkg in REQUIRED:
        try:
            importlib.import_module(pkg)
        except ImportError:
            print(f"Устанавливаю {pkg}...")
            subprocess.check_call([sys.executable, '-m', 'pip', 'install', pkg])

ensure_dependencies()

# ------------------------------
# Параметры модели
# ------------------------------

@dataclass
class ModelParams:
    lambda_: np.ndarray = field(default_factory=lambda: np.array([0.5]*6))
    alpha: np.ndarray = field(default_factory=lambda: np.array([0.2]*6))
    beta: np.ndarray = field(default_factory=lambda: np.array([0.1]*6))
    gamma: np.ndarray = field(default_factory=lambda: np.array([0.1]*6))
    delta: np.ndarray = field(default_factory=lambda: np.array([0.05]*6))
    theta: np.ndarray = field(default_factory=lambda: np.array([0.3]*6))
    tau_E: float = 2.0

# ------------------------------
# Ядро модели
# ------------------------------
```

```
class LivingEquilibriumModel:
    def __init__(self, params: ModelParams):
        self.p = params

    def errors(self, x):
        return np.maximum(0, self.p.theta - x)

    def derivative(self, t, x):
        S, O, M, P, A, D = x
        lam, alpha, beta, gamma, delta, theta = (
            self.p.lambda_, self.p.alpha, self.p.beta,
            self.p.gamma, self.p.delta, self.p.theta
        )
        e = self.errors(x)
        x_bar = np.mean(x[:5])

        dS = -lam[^0]*S + alpha[^0]*e[^0] + beta[^0]*O*M - gamma[^0]*P - delta[^0]*(1
        dO = -lam[^1]*O + alpha[^1]*e[^1] + beta[^1]*S*M - gamma[^1]*P - delta[^1]*(1
        dM = -lam[^2]*M + alpha[^2]*e[^2] + beta[^2]*S*O - gamma[^2]*P - delta[^2]*(1
        dP = -lam[^3]*P + alpha[^3]*e[^3] + beta[^3]*S*O*M - gamma[^3]*A - delta[^3]
        dA = -lam[^4]*A + alpha[^4]*e[^4] + beta[^4]*P - gamma[^4]*(S+O+M) + delta[^4
        dD = -lam[^5]*D + alpha[^5]*e[^5] + beta[^5]*(S+O+M) - gamma[^5]*x_bar

        return [dS, dO, dM, dP, dA, dD]

# ------------------------------
# Запуск симуляции
# ------------------------------

def run_simulation():
    params = ModelParams()
    model = LivingEquilibriumModel(params)
    x0 = np.array([0.8, 0.8, 0.8, 0.8, 0.8, 0.8])
    t_span = (0, 20)
    t_eval = np.linspace(*t_span, 500)

    sol = solve_ivp(model.derivative, t_span, x0, t_eval=t_eval, method='RK45')
    return sol

# ------------------------------
# Сохранение в CSV
# ------------------------------

def save_results_to_csv(sol, filename='output/model_results.csv'):
    df = pd.DataFrame({
        'time': sol.t,
        'S': sol.y[^0],
        'O': sol.y[^1],
        'M': sol.y[^2],
        'P': sol.y[^3],
        'A': sol.y[^4],
        'D': sol.y[^5],
    })
    df.to_csv(filename, index=False)
    print(f"Результаты сохранены в {filename}")
```

```
# ------------------------------
# Визуализация
# ------------------------------

def plot_2d(sol):
    labels = ['S', 'O', 'M', 'P', 'A', 'D']
    plt.figure(figsize=(12, 6))
    for i in range(6):
        plt.plot(sol.t, sol.y[i], label=labels[i])
    plt.axhline(0, color='r', linestyle='--')
    plt.xlabel('Время')
    plt.ylabel('Значение')
    plt.title('Динамика модели живого равновесия')
    plt.legend()
    plt.grid(True)
    plt.savefig('output/model_dynamics_2d.png')
    plt.show()

def plot_3d(sol):
    fig = plt.figure()
    ax = fig.add_subplot(111, projection='3d')
    ax.plot(sol.y[^0], sol.y[^1], sol.y[^2], lw=2)
    ax.set_xlabel('S')
    ax.set_ylabel('O')
    ax.set_zlabel('M')
    ax.set_title('3D фазовый портрет: S-O-M')
    plt.savefig('output/phase_portrait_3d.png')
    plt.show()

def animate(sol):
    labels = ['S', 'O', 'M', 'P', 'A', 'D']
    fig, ax = plt.subplots()
    lines = [ax.plot([], [], label=label)[^0] for label in labels]
    ax.set_xlim(0, len(sol.t))
    ax.set_ylim(0, 1)
    ax.legend()

    def update(frame):
        for i, line in enumerate(lines):
            line.set_data(sol.t[:frame], sol.y[i][:frame])
        return lines

    ani = animation.FuncAnimation(fig, update, frames=len(sol.t), interval=50, blit=T
    plt.show()

# ------------------------------
# Ключевые выводы
# ------------------------------

def print_key_findings(sol):
    print("\n=== КЛЮЧЕВЫЕ ВЫВОДЫ ===")
    print("1. Все контуры остаются выше нуля — система не разрушается полностью.")
    print("2. Адаптация (A) усиливается при снижении других контуров.")
    print("3. Разнообразие (D) падает при усилении усреднения.")
    print("4. Процедура (P) — критический контур: при её ослаблении система деградиру
```

```
    print("5. Модель показывает устойчивость при малых возмущениях и уязвимость при 

# ----------------------------
# Главная функция
# ----------------------------

def main():
    os.makedirs('output', exist_ok=True)
    sol = run_simulation()

    # Сохранение в CSV
    save_results_to_csv(sol, 'output/model_results.csv')

    plot_2d(sol)
    plot_3d(sol)
    animate(sol)
    print_key_findings(sol)

if __name__ == "__main__":
    main()
```

Что изменится

- После запуска появится файл `output/model_results.csv`.
- В нём будут колонки: `time`, `S`, `O`, `M`, `P`, `A`, `D`.
- Файл можно открыть в Excel, LibreOffice, Google Sheets или pandas.

Финальная формулировка

> **CSV — это не просто «файл», а способ сделать модель проверяемой: результаты можно проанализировать, передать, визуализировать заново.**

Это и есть практическая реализация: не «философия на бумаге», а **симуляция с сохраняемыми данными.** [3] [4] [1]

⁂

1. https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.to_csv.html
2. https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.to_csv.html
3. https://numpy.org/numpy-tutorials/save-load-arrays/
4. https://numpy.org/doc/stable/reference/generated/numpy.savetxt.html
5. https://pandas.pydata.org/pandas-docs/version/0.22/generated/pandas.DataFrame.to_csv.html
6. https://pandas.pydata.org/docs/dev/reference/api/pandas.DataFrame.to_csv.html
7. https://pandas.pydata.org/pandas-docs/stable/dev/reference/api/pandas.DataFrame.to_csv.html
8. https://pandas.pydata.org/pandas-docs/version/3.1/reference/api/pandas.DataFrame.to_csv.html
9. http://pythongis.org/part1/chapter-03/nb/00-pandas-basics.html
10. https://pandas.pydata.org/pandas-docs/version/1.5/reference/api/pandas.Series.to_csv.html

11. https://numpy.org/doc/stable//reference/generated/numpy.save.html
12. https://numpy.org/devdocs/reference/generated/numpy.savetxt.html
13. https://numpy.org/doc/stable/reference/generated/numpy.savetxt.html?highlight=savetxt
14. https://www.annenbergclassroom.org/glossary_term/majority-rule-and-minority-rights/
15. https://www.britannica.com/topic/veche
16. https://www.victorianweb.org/victorian/philosophy/mill/liberty.html
17. https://en.wikipedia.org/wiki/On_Liberty